

JENSON USA

project

Presented by Saransh Garg

ABOUT

Jenson USA is a well-known online retailer for cyclists, offering a wide range of bikes, parts, and gear. They're popular for their excellent customer service, competitive prices, and fast shipping. Whether you're into road cycling, mountain biking, or just need a new accessory, Jenson USA has a solid selection and helpful advice to keep you riding smoothly. Their "Ride It. Love It. Guarantee" makes trying new products worry-free, which customers really appreciate!

1 Find the total number of products sold by each store along with the store name .

```
SELECT stores.store_name, SUM(order_items.quantity) AS total_quantity
      FROM orders
      JOIN order_items ON order_items.order_id = orders.order_id
      JOIN stores ON stores.store_id = orders.store_id
      GROUP BY stores.store_name;
```

2

Calculate the cumulative sum of quantities sold for each product over time.

```
SELECT product_id, order_date, quantity,  
SUM(quantity) OVER (PARTITION BY product_id ORDER BY order_date)  
FROM (SELECT order_items.product_id,  
            orders.order_date,  
            SUM(order_items.quantity) AS quantity  
      FROM orders  
      JOIN order_items ON orders.order_id = order_items.order_id  
      GROUP BY order_items.product_id, orders.order_date) a;
```


3 Find the product with the highest total sales (quantity * price) for each category.

```
WITH a AS (SELECT categories.category_id, categories.category_name,  
                products.product_id, products.product_name,  
                SUM(order_items.quantity * (order_items.list_price - order_items.discount)) AS sales  
            FROM order_items  
            JOIN products ON products.product_id = order_items.product_id  
            JOIN categories ON categories.category_id = products.category_id  
            GROUP BY categories.category_id, categories.category_name,  
                     products.product_id, products.product_name)  
SELECT * FROM  
(SELECT *, RANK() OVER (PARTITION BY category_id ORDER BY sales DESC) AS rnk  
 FROM a) b  
WHERE rnk = 1;
```

4

Find the customer who spent the most money on orders.

```
WITH a AS (SELECT customers.customer_id,  
  CONCAT(customers.first_name," ", customers.last_name) AS full_name,  
  SUM(order_items.quantity * (order_items.list_price - order_items.discount)) AS sales  
  FROM customers  
  JOIN orders ON customers.customer_id = orders.customer_id  
  JOIN order_items ON order_items.order_id = orders.order_id  
  GROUP BY customers.customer_id,  
  CONCAT(customers.first_name," ", customers.last_name))  
  SELECT * FROM  
  (SELECT *, RANK() OVER (ORDER BY sales DESC) AS rnk  
   FROM a) b  
  WHERE rnk = 1;
```

5

Find the highest-priced product for each category name.

```
SELECT * FROM  
(SELECT categories.category_id, categories.category_name,  
        products.product_name, products.list_price,  
RANK() OVER (PARTITION BY categories.category_id ORDER BY products.list_price DESC) AS rnk  
FROM products  
JOIN categories ON products.category_id = categories.category_id) a  
WHERE rnk = 1;
```

6

Find the total number of orders placed by each customer per store.

```
SELECT stores.store_name, CONCAT(customers.first_name, " ", customers.last_name) AS full_name,  
       COUNT(orders.order_id) AS order_count  
       FROM orders  
       JOIN stores ON orders.store_id = stores.store_id  
       JOIN customers ON orders.customer_id = customers.customer_id  
GROUP BY stores.store_name, CONCAT(customers.first_name, " ", customers.last_name);
```

7

Find the names of staff members who have not made any sales.

```
SELECT staffs.staff_id,  
CONCAT(staffs.first_name," ", staffs.last_name) AS full_name  
FROM staffs  
WHERE NOT EXISTS (SELECT staff_id FROM orders  
WHERE orders.staff_id = staffs.staff_id);
```


8

Find the top 3 most sold products in terms of quantity.

```
SELECT product_name FROM
(SELECT products.product_id, product_name, SUM(order_items.quantity) AS quantity,
RANK() OVER (ORDER BY SUM(order_items.quantity) DESC) AS rnk
FROM products
JOIN order_items ON products.product_id = order_items.product_id
GROUP BY products.product_id, product_name)a
WHERE rnk <= 3;
```

```
WITH m AS (SELECT list_price,  
ROW_NUMBER() OVER (ORDER BY list_price) AS rn,  
COUNT(list_price) OVER () AS cn  
FROM order_items)  
SELECT CASE  
WHEN cn % 2 = 0 THEN (SELECT AVG(list_price) FROM m WHERE rn IN (cn/2, (cn/2) + 1))  
ELSE (SELECT list_price FROM m WHERE rn = (cn + 1)/2)  
END AS median from m limit 1;
```

10

List all products that have never been ordered.(use Exists)

```
SELECT products.product_id,  
       products.product_name  
FROM products  
WHERE NOT EXISTS (SELECT product_id FROM order_items  
                  WHERE order_items.product_id = products.product_id);
```

11

List the names of staff members who have made more sales than the average number of sales by all staff members.

```
SELECT staffs.staff_id,  
COALESCE(SUM(order_items.quantity * (order_items.list_price - order_items.discount)), 0) AS sales  
FROM orders  
RIGHT JOIN staffs ON staffs.staff_id = orders.staff_id  
LEFT JOIN order_items ON orders.order_id = order_items.order_id  
GROUP BY staffs.staff_id  
HAVING SUM(order_items.quantity * (order_items.list_price - order_items.discount)) >  
(SELECT AVG(sales) FROM  
(SELECT staffs.staff_id,  
COALESCE(SUM(order_items.quantity * (order_items.list_price - order_items.discount)), 0) AS sales  
FROM orders  
RIGHT JOIN staffs ON staffs.staff_id = orders.staff_id  
LEFT JOIN order_items ON orders.order_id = order_items.order_id  
GROUP BY staffs.staff_id) AS c);
```

12

Identify the customers who have ordered all types of products
(i.e., from every category).

```
SELECT customers.customer_id
FROM customers
JOIN orders ON customers.customer_id = orders.customer_id
JOIN order_items ON orders.order_id = order_items.order_id
JOIN products p ON order_items.product_id = p.product_id
GROUP BY customers.customer_id
HAVING COUNT(DISTINCT p.category_id) = (SELECT COUNT(category_id) FROM categories);
```


THANK YOU

for your time and attention

*Please Like, Share & Do
Comment .*